Μάθημα	**Ανάλυση Δεδομένων μεγάλου Όγκου (Big Data)**
Αριθμός Εργασίας	Πρώτη Ομαδική εργασία
Είδος εργασίας	**Υποχρεωτική**
Μορφή Υλοποίησης	**Ομαδική** (1 έως και 2 άτομα)
Ημερομηνία Παράδοσης	**Παρασκευή 28 Νοεμβρίου 2025 23:59** (Δε θα δοθεί παράταση)
Παράδοση εργασίας	**e-class**

Flight Delay Analytics με Apache Spark

Περιγραφή Προβλήματος

Στόχος είναι η **ανάλυση δεδομένων καθυστερήσεων πτήσεων** χρησιμοποιώντας το Apache Spark. Θα αξιοποιηθεί το dataset flights_2000.csv, το οποίο περιλαμβάνει πληροφορίες για πτήσεις, αεροδρόμια και καθυστερήσεις αναχωρήσεων. Κάθε εγγραφή περιέχει πεδία όπως: FL_DATE, ORIGIN_AIRPORT, DEST_AIRPORT, DEP_DELAY, ARR_DELAY, CANCELLED, AIRLINE. Οι φοιτητές καλούνται να υλοποιήσουν δύο διαφορετικές προσεγγίσεις ανάλυσης: με Spark RDD API και Spark DataFrame API και να αξιολογήσουν επιδόσεις, εκφραστικότητα και αποδοτικότητα των δύο προσεγγίσεων.

Περιγραφή δεδομένων

Για την παρούσα εργασία θα χρησιμοποιηθεί το αρχείο **flights_2000.csv**, το οποίο περιέχει πραγματικά δεδομένα πτήσεων (2000 γραμμές). Κάθε γραμμή του αρχείου αντιστοιχεί σε μία πτήση και περιλαμβάνει βασικές πληροφορίες σχετικά με την ημερομηνία, την αεροπορική εταιρεία, το αεροδρόμιο αναχώρησης και προορισμού, καθώς και τους χρόνους καθυστέρησης. (Μπορείτε να κατεβάσετε το αρχείο από το ακόλουθο Link). Το αρχείο **flights_2000.csv** περιέχει εγγραφές πτήσεων με τα ακόλουθα πεδία:

- **FL_DATE**: Ημερομηνία εκτέλεσης της πτήσης. Μπορεί να χρησιμοποιηθεί για ομαδοποίηση ή φιλτράρισμα ανά χρονική περίοδο.
- **AIRLINE**: Κωδικός αεροπορικής εταιρείας (π.χ. WN, DL, UA). Μπορεί να χρησιμοποιηθεί για σύγκριση επιδόσεων μεταξύ εταιρειών.
- **FL_NUM**: Αριθμός πτήσης. Συνδυάζεται με το AIRLINE για μοναδικό αναγνωριστικό πτήσης.
- **ORIGIN_AIRPORT**: Κωδικός αεροδρομίου αναχώρησης (International Air Transport Association (IATA): διεθνής τριψήφιος κωδικός αεροδρομίου, π.χ. ATH για Αθήνα, LHR για Λονδίνο). Κεντρικό πεδίο για υπολογισμό καθυστερήσεων ανά αεροδρόμιο.
- **DEST_AIRPORT**: Κωδικός αεροδρομίου προορισμού (IATA). Συνδυάζεται με το ORIGIN_AIRPORT για ανάλυση διαδρομών (routes).
- **SCHED_DEP**: Προγραμματισμένη ώρα αναχώρησης. Από το πεδίο αυτό μπορεί να εξαχθεί η ώρα (HOUR) για ανάλυση καθυστερήσεων ανά ώρα.
- **DEP_DELAY**: Καθυστέρηση αναχώρησης σε λεπτά. Αρνητικές τιμές υποδηλώνουν πρόωρη αναχώρηση, null = απουσία δεδομένων.
- **SCHED_ARR** : Προγραμματισμένη ώρα άφιξης. Δεν χρησιμοποιείται άμεσα, αλλά μπορεί να βοηθήσει σε ελέγχους διάρκειας πτήσης.
- **ARR_DELAY**: Καθυστέρηση άφιξης σε λεπτά. Μπορεί να αξιοποιηθεί για έλεγχο συνέπειας με το DEP_DELAY.
- **DIST_KM**: Απόσταση πτήσης σε χιλιόμετρα. Επιτρέπει συγκρίσεις καθυστέρησης σε σχέση με το μήκος διαδρομής.
- **CANCELLED:** Ένδειξη αν η πτήση ακυρώθηκε (0 = εκτελέστηκε, 1 = ακυρώθηκε). Οι ακυρωμένες πτήσεις πρέπει να εξαιρούνται από τους υπολογισμούς καθυστερήσεων.

Θέμα 1: Spark RDD API (3 βαθμοί)

1) Διαβάστε το αρχείο Flights.csv ως text RDD (textFile ()), κάντε split/parse.
2) Εξαγωγή ζευγών: (origin_airport, delay_minutes). Μπορείτε να χρησιμοποιήσετε τη μέθοδο map() για να εξαγάγετε ζεύγη (origin_airport, delay_minutes).
3) Υπολογίστε μέση καθυστέρηση ανά αεροδρόμιο αναχώρησης με combineByKey ή reduceByKey (άθροισμα, πλήθος → average).
4) Επιστρέψτε top-10 αεροδρόμια με μεγαλύτερη μέση καθυστέρηση (descending).
5) Εκτελέστε το πρόγραμμα συνολικά πέντε (5) φορές.
   a. Αφαιρέστε τη μεγαλύτερη και τη μικρότερη τιμή χρόνου και υπολογίστε τον μέσο όρο των τριών (3) ενδιάμεσων εκτελέσεων.
   b. Καταγράψτε:
      i. τον συνολικό χρόνο εκτέλεσης (από την εκκίνηση μέχρι την παραγωγή του αποτελέσματος),
      ii. Τον χρόνο του πρώτου action, δηλαδή τη χρονική στιγμή όπου το Spark εκτελεί πραγματικά τον υπολογισμό (λόγω της lazy evaluation). Αναφέρετε τους μέσους χρόνους σε πίνακα στην αναφορά σας.

Παράδοση: sample output (top-10) και χρόνοι.

Θέμα 2: Spark DataFrame API (3 βαθμοί)

1) Διαβάστε το αρχείο **Flights.csv** ως **DataFrame** με Spark.
2) Εμφανίστε το **schema** του πίνακα με printSchema() για να επαληθεύσετε τη δομή.
3) Φιλτράρετε τις εγγραφές όπου CANCELLED = 0 και αγνοήστε κενές ή μη αριθμητικές τιμές καθυστέρησης (π.χ. μέσω df.na.drop()). Υπολογίστε:
   a. τη **μέση καθυστέρηση αναχώρησης** ανά διαδρομή (origin–destination) χρησιμοποιώντας groupBy()
   b. τα **10 routes με τη μεγαλύτερη μέση καθυστέρηση**, ταξινομημένα κατά φθίνουσα σειρά.
4) Αποθηκεύστε το αποτέλεσμα σε αρχείο CSV με write.csv(...)
5) Εκτελέστε το πρόγραμμα συνολικά **πέντε (5) φορές** και:
   a. Αφαιρέστε τη μεγαλύτερη και τη μικρότερη τιμή χρόνου εκτέλεσης.
   b. Υπολογίστε τον μέσο όρο των τριών (3) ενδιάμεσων εκτελέσεων.
   c. Καταγράψτε τον **συνολικό χρόνο εκτέλεσης** και τον **χρόνο πρώτου action**, όπου το Spark εκτελεί πραγματικά τον υπολογισμό (lazy evaluation). Αναφέρετε τους μέσους χρόνους σε πίνακα στην αναφορά σας.

Παράδοση: δείγμα εξόδου (top-10 routes) και πίνακας χρόνων εκτέλεσης.

Θέμα 3: Συγκριτική Ανάλυση RDD vs DataFrame (2 βαθμοί)

1) Συγκρίνετε τις δύο υλοποιήσεις ως προς:
   a. ευκολία υλοποίησης,
   b. χρόνο εκτέλεσης,
   c. εκφραστικότητα κώδικα,
   d. δυνατότητες βελτιστοποίησης (Catalyst optimizer).
2) Παρουσιάστε σύντομο σχολιασμό στην αναφορά σας (μέγιστο ½ σελίδας) συνοδευόμενο από τον πίνακα χρόνων εκτέλεσης.
3) Εξηγήστε, με βάση τα αποτελέσματά σας, ποια προσέγγιση θεωρείτε καταλληλότερη για μεγάλα σύνολα δεδομένων και γιατί.

Παράδοση: κείμενο σχολιασμού και πίνακας χρόνων στη συνοδευτική αναφορά.

Θέμα 4: Οπτικοποίηση & Ανάλυση (1 βαθμός)

1) Μετατρέψτε το τελικό αποτέλεσμα του DataFrame σε **Pandas DataFrame**.
2) Δημιουργήστε γράφημα με τη βιβλιοθήκη **matplotlib**.
3) Εμφανίστε τα **10 routes με τη μεγαλύτερη μέση καθυστέρηση** (οριζόντιο γράφημα).

Παράδοση: γράφημα και σύντομος σχολιασμός αποτελεσμάτων.

Θέμα 5: Επέκταση / Εμπλουτισμός Ανάλυσης (1 βαθμός)

1) Δημιουργήστε **πρόσθετη ανάλυση ή οπτικοποίηση** με βάση τα ίδια δεδομένα. Για τα παρακάτω:
   a. Μέση καθυστέρηση **ανά ώρα αναχώρησης** (withColumn("HOUR", (col("SCHED_DEP").substr(12,2)).cast("int")))
   b. **Κατανομή καθυστερήσεων** ανά αεροπορική εταιρεία ή μήνα.
   c. **Heatmap** routes vs delay.
2) Εξηγήστε σύντομα τον σκοπό της επέκτασης και το τι αναδεικνύει.
3) Ενσωματώστε το γράφημα ή τον πίνακα στην αναφορά σας, μαζί με σύντομο σχολιασμό.

Παράδοση: ένα επιπλέον γράφημα ή/και πίνακας με σχολιασμό.

Οδηγίες & Παραδοτέα

1) **Γλώσσα / Περιβάλλον**
   a. Υλοποίηση σε **PySpark (Python 3.x, Spark 3.x)**.
   b. Επιτρέπεται εναλλακτικά **Scala/Spark**, εφόσον τηρηθούν οι ίδιοι κανόνες μετρήσεων.
   c. Επιτρεπόμενα περιβάλλοντα: **Colab / Jupyter Notebook / spark-submit**. (Προτεινόμενο για ευκολία Colab)
2) **Τι παραδίδετε (όλα σε ένα .zip)**
   a. **Κώδικας**:
      i. flight_delay_spark.ipynb (σε περίπτωση που δουλέψετε σε colab share και το notebook ή .scala/.py αν δουλέψατε εκτός notebook).
      ii. Αν υπάρχουν βοηθητικά modules: φάκελος src/ και **README** με οδηγίες εκτέλεσης.
   b. **Αποτελέσματα**:
      i. Φάκελος out/ με τα CSV από το Μέρος DataFrame (top-10 routes).
      ii. Το γράφημα(τα) του Θέματος 3 σε μορφή εικόνας (.png ή .pdf).
   c. **Αναφορά (PDF, 2–3 σελ.)** που περιλαμβάνει:
      i. σύντομη **Περιγραφή/Καθαρισμό** δεδομένων,
      ii. **Πίνακες με χρόνους** εκτέλεσης (RDD & DataFrame, 5 runs ⇒ drop max/min ⇒ μέσος όρος 3),
      iii. **Σύγκριση RDD vs DataFrame** (ευκολία, χρόνοι, εκφραστικότητα, Catalyst),
      iv. τα **γραφικά** με συνοπτικό σχολιασμό.
   d. **(Προαιρετικά)** αρχείο requirements.txt ή environment.yml αν χρειάζονται πρόσθετες βιβλιοθήκες.
3) **Κανόνες μετρήσεων (εφαρμόζονται και στα δύο μέρη)**
   a. Εκτελέστε **5 φορές** το ίδιο σενάριο, αφαιρέστε **μεγαλύτερη/μικρότερη** τιμή και αναφέρετε **μέσο όρο** των 3.
   b. Καταγράψτε **συνολικό χρόνο εκτέλεσης** και **χρόνο πρώτου action** (λόγω lazy evaluation).
   c. Δηλώστε **περιβάλλον εκτέλεσης** (CPU/GPU, μνήμη, εκδόσεις Python/Spark).
   d. Μην κάνετε collect() σε μεγάλα αποτελέσματα. Χρησιμοποιήστε show() ή write.csv.

4) **Ονοματοδοσία & Υποβολή**
   a. Όνομα αρχείου: uom_bigdata_2025_hw1_<Surname1_Surname2>.zip.
   b. Υποβολή **ηλεκτρονικά στο e-class** μέχρι την καταληκτική ημερομηνία. Μία υποβολή ανά ομάδα.
   c. Οι ομάδες θα δηλωθούν εκ των υστέρων πριν την υποβολή εώς και μία εβδομάδα πριν την υποβολή, ήτοι 21/11/2025.
5) **Ακεραιότητα / Αναπαραγωγιμότητα**
   a. Η εργασία είναι **ομαδική (1–2 άτομα)**. **Δηλώστε ονόματα/ΑΜ στην αρχή του notebook & στην αναφορά.**
   b. Ο κώδικας πρέπει να **τρέχει αυτούσιος** (relative paths, όχι προσωπικά paths).
   c. Αναφέρετε οποιαδήποτε εξωτερική πηγή χρησιμοποιήθηκε (κώδικας/ιδέες/ ChatGPT/ BlackBox AI και πως το χρησιμοποιήσατε).
   d. Αναφέρετε οποιαδήποτε συνεργασία με άλλη/άλλες ομάδες.

Ενδεικτικές Αναφορές

[1] **Leskovec, J., Rajaraman, A., Ullman, J. D. (2020).** *Mining of Massive Datasets* (3rd Edition). http://www.mmds.org/
[2] Lin, J. & Dyer, C. (2010). Data-Intensive Text Processing with MapReduce. Διαθέσιμο online: https://lintool.github.io/MapReduceAlgorithms/
[3] RDD Programming Guide - Spark 4.0.1 Documentation
[4] Spark SQL, DataFrames and Datasets Guide
[5] Διαλέξεις, σημειώσεις και ασκήσεις μαθήματος.